Quantum Lexicographically Minimal String Rotation and Benzenoid Identification via QRAM-Accelerated Search

Mark Boady Gary Pham

March 9, 2026

Abstract

The lexicographically minimal string rotation (LMSR) finds the cyclic rotation of a string that is the smallest in lexicographic order. Classical solutions such as Booth's algorithm solve LMSR in linear time. Certain applications in bioinformatics and cheminformatics, such as benzenoid identification, can benefit from quantum speedups for repeated pattern searches. A quantum algorithm is proposed that leverages Grover-based search over all rotations combined with Quantum Random Access Memory (QRAM) for efficient access to superposition values. Its asymptotic performance is analyzed. Applications to generating canonical benzenoid representations are discussed. In ideal QRAM, our algorithm achieves a worst-case run-time of $O(\sqrt{N}\log N)$ for an input string of length N, compared to the classical $O(N)$ bound. Although near-term hardware limits the immediate quantum advantage, QRAM-accelerated search holds promise for large-scale string-based identification tasks in computational chemistry.

1 Introduction

The lexicographically minimal string rotation (LMSR) problem arises in various areas of computer science, including text processing, bioinformatics, and data compression. Given a string S of length N, LMSR asks for the rotation of S that is lexicographically smallest among all possible rotations. Classical algorithms such as Booth's algorithm solve LMSR in $O(N)$ time [1], while Shiloach's fast canonization method provides an alternative linear-time approach [2]. When strings encode molecular structures, finding the canonicalization across thousands or millions of candidate strings is required. This happens when benzenoid chemicals are represented by boundary-edge codes. In cheminformatics, canonical representations facilitate isomorphism testing and database lookups. Benzenoids can be large and numerous. This means that a quantum acceleration of repeated LMSR computations can yield practical benefits.

Quantum computers offer quadratic speedups for unstructured search via Grover's algorithm [3, 5], building on the broader paradigm of quantum computational ad-

vantage first demonstrated by algorithms such as Shor's factoring algorithm [4]. LMSR search can be implemented by querying a superposition over all rotations. To compare two rotated versions of a string, one must access individual characters coherently. A mechanism is required to index the positions of the strings in superposition. Quantum Random Access Memory (QRAM) can serve this role [8]. A quantum LMSR algorithm is proposed that uses QRAM to load the input string into quantum memory. It applies a Grover-like subroutine to find the smallest rotation, and achieves a worst-case runtime of $O\big(\sqrt{N}\log N\big)$ under an ideal QRAM model.

This paper is organized as follows. In Section 2, a survey of classical and quantum search algorithms, QRAM architectures, and previous work on LMSR and benzenoid canonicalization is presented. Section 3 provides necessary preliminaries, including notation and a summary of the classical LMSR problem. Section 4 details our quantum algorithm - its circuit structure, pseudocode, and example walkthroughs. Section 5 discusses QRAM optimizations, including binary and hashing schemes. Section 6 presents runtime and resource analyses. In Section 7, shares empirical results comparing classical and quantum runtimes (simulated) on benchmark strings and benzenoid boundary codes. Section 8 concludes with future directions for near-term implementation and extensions to related graph-isomorphism problems.

2 Background

2.1 Algorithmic Foundations

Classical solutions to the LMSR problem include Booth's algorithm, which identifies the lexicographically minimal rotation in $O(N)$ time by maintaining two indices and performing sequential character comparisons [1]. In contrast, quantum search algorithms such as Grover's algorithm offer a quadratic speedup for finding marked items in an unstructured database of size N in $O(\sqrt{N})$ time [3, 5]. Dürr and Høyer extended Grover's framework to locate the minimum element in an unsorted list in $O(\sqrt{N})$ time without requiring explicit pairwise comparisons at runtime [6]. Our approach directly employs this Dürr–Høyer minimum-finding subroutine on the set of rotation-cost values, ensuring that each iteration yields the globally minimal rotation index by construction rather than comparing two rotations explicitly.

To access string characters in superposition, we rely on Quantum Random Access Memory (QRAM) architectures, which permit coherent indexing and data retrieval in $O(\log N)$ depth [8–10]. Unlike the Tower framework—which processes lists but cannot retrieve arbitrary values in superposition—QRAM supports parallel lookups of all characters needed to evaluate rotation costs [15]. Prior quantum proposals for LMSR have focused on variants of lexicographic comparison circuits. Our algorithm avoids repeated "smaller-than" checks between rotation-string pairs by leveraging Dürr–Høyer's minimum finding. Instead, it treats each rotation as an item in an unsorted list whose cost can be evaluated via a QRAM-supported oracle.

Previous efforts applying quantum algorithms to string problems include techniques for quantum string matching [12] and quantum pattern matching [13]. Quantum walk-based approaches have also been applied to element distinctness and related search problems [14]. Explicit quantum algorithms addressing LMSR appear limited. Tower [15] is a framework for data structures in quantum superposition, but does not allow randomized superposition-based comparisons of list elements. It focuses on fixed-width list indexing. Our approach differs by directly leveraging QRAM and Grover-type minimum finding to identify the minimal rotation.

2.2 Benzenoid Graphs and Boundary Codes

Benzenoids are planar polycyclic aromatic hydrocarbons composed of fused benzene rings. Their structure can be represented by hexagonal tilings. Canonicalization is essential for isomer enumeration, property prediction, and database searches. The mathematical theory of benzenoid hydrocarbons has been developed extensively [17–19]. One popular representation uses Boundary Edge Codes (BEC), which encode the perimeter of a benzenoid as a cyclic sequence of edge angles and lengths. Each benzenoid yields a cyclic string over a finite alphabet (e.g., $\{1, 2, 3\}$ indicating edge directions). To compare two benzenoids for equivalence, one must find a canonical rotation of their boundary code—again reducing to the LMSR problem.

Recent work on quantum approaches to computational chemistry has explored using quantum algorithms for molecular structure determination and isomer identification [20]. QRAM-accelerated LMSR provides a direct method for establishing a unique identifier for each benzenoid structure. By computing the minimal cyclic rotation of the BEC, one obtains a canonical signature that is comparable in classical and quantum settings.

3 Preliminaries

In this section, we introduce notation, recall the classical LMSR problem, and outline our quantum model—including QRAM assumptions—that underlie the algorithm described in Section 4.

3.1 Notation and Definitions

Let $S = s_0 s_1 \dots s_{N-1}$ be a string of length N over a totally ordered alphabet Σ. For $0 \le k < N$, define the k-th cyclic rotation

$$S^{(k)} \;=\; s_k\, s_{k+1}\, \dots\, s_{N-1}\, s_0\, s_1\, \dots\, s_{k-1}.$$

The lexicographically minimal rotation index is

$$\mathrm{LMSR}(S) \;=\; \arg\min_{0 \le k < N} \bigl\{\, S^{(k)} \bigr\},$$

where comparisons are taken in lexicographic order. We will refer to the "rotation cost" of index k as an integer value that encodes the lexicographic rank of $S^{(k)}$ among all rotations—i.e., smaller cost means a lexicographically smaller rotation.

3.2 Classical LMSR via Booth's Algorithm

Booth's algorithm finds LMSR(S) in $O(N)$ time using two pointer indices i and j (initialized to 0 and 1) and a shift offset $k = 0$. At each step:

1. Compare characters $S[(i + k) \bmod N]$ and $S[(j + k) \bmod N]$.
2. If equal, increment k; if one is smaller, advance the other pointer by $k+1$ and reset $k = 0$.
3. Terminate when one pointer reaches N. Then $\min(i, j)$ is the minimal rotation index.

This method scans each character at most twice and uses $O(1)$ extra space. For large or repeated LMSR queries—such as canonicalizing many benzenoid boundary codes—an $O(N)$ scan per string can become a significant bottleneck.

3.3 Quantum Model and QRAM Assumptions

We assume a fault-tolerant circuit model [16] with access to Quantum Random Access Memory (QRAM). Define an address register of $\lceil \log_2 N \rceil$ qubits and a data register that can store one alphabet symbol $|s_i\rangle$. A single QRAM access implements

$$|i\rangle_{\text{addr}}|0\rangle_{\text{data}} \xrightarrow{\text{QRAMRead}} |i\rangle_{\text{addr}}|s_i\rangle_{\text{data}},$$

in $O(\log N)$ depth under an idealized bucket-brigade architecture [9]. We ignore error-correction overheads but account for asymptotic costs. Address registers may be prepared in uniform superposition, enabling simultaneous access to all string positions when evaluating rotation costs.

Our quantum algorithm treats each rotation index $k \in \{0, \ldots, N-1\}$ as an item in an unsorted list. Instead of performing explicit pairwise lexicographic comparisons, we define a cost-evaluation oracle $\mathcal{O}_{\text{cost}}$ that, given $|k\rangle$, coherently computes an integer cost corresponding to the lexicographic rank of $S^{(k)}$. This cost can be derived by comparing $S^{(k)}$ against a reference value in superposition or by accumulating a penalty for each mismatch position via QRAM reads. Once costs are available, we apply Dürr–Høyer's minimum finding [6] directly to these costs, finding the minimal rotation index in $O(\sqrt{N} \log N)$ expected time under QRAM.

Finally, we assume QRAM overheads scale as follows:

- **Bucket-Brigade QRAM:** $O(\log N)$ depth per access with $O(N)$ switch qubits.

- **Hashing-Based QRAM (optional):** Expected $O(\log N)$ depth per access using 2-universal hashing, with collision resolution costs in expectation constant.

Under these assumptions, each cost evaluation (which may involve up to N character accesses in the worst case) costs $O(N \log N)$ depth, while early-exit heuristics on random or non-degenerate inputs reduce expected cost to $O(\log N)$. In Section 6, we analyze worst-case and average-case complexities under these models.

4 Algorithm Description

In this section, we present our quantum LMSR algorithm, which applies Dürr–Høyer minimum finding directly to the set of rotation-cost values rather than performing explicit pairwise string comparisons. By encoding each rotation's cost—computed via QRAM-supported character lookups—into an oracle, the Dürr–Høyer subroutine identifies the minimal rotation index in $O(\sqrt{N} \log N)$ expected time. We first describe how the input string is loaded into QRAM (Section 4.1), then explain the cost-evaluation oracle that avoids explicit "smaller-than" checks (Section 4.2). Finally, we outline the overall minimum-finding routine using amplitude amplification and walk through a concrete example (Section 4.3).

4.1 Data Loading into QRAM

1. **Initialize QRAM:** Given the input string $S = s_0 s_1 \dots s_{N-1}$, allocate a QRAM array of N memory cells, each capable of storing one symbol of Σ.

2. **Populate Data Registers:** For each $i \in \{0, \dots, N-1\}$, write the classical symbol s_i into QRAM address $|i\rangle$. After this step, the QRAM satisfies

$$|i\rangle_{\text{addr}}|0\rangle_{\text{data}} \xrightarrow{\text{QRAMRead}} |i\rangle_{\text{addr}}|s_i\rangle_{\text{data}} \quad \text{for all } i.$$

3. **Prepare Index Superposition:** Allocate an $\lceil \log_2 N \rceil$-qubit index register and initialize it to the uniform superposition

$$\frac{1}{\sqrt{N}} \sum_{k=0}^{N-1} |k\rangle_{\text{idx}}.$$

   This register will enumerate candidate rotations k.

4.2 Cost-Evaluation Oracle

We define an oracle $\mathcal{O}_{\text{cost}}$ that, given a superposed index $|k\rangle_{\text{idx}}$, computes an integer "rotation cost" $C(k)$ proportional to the lexicographic rank of $S^{(k)}$. Importantly,

we never compare two rotations explicitly; instead, each k is assigned a cost by counting mismatches against a fixed reference (e.g., the lexicographically smallest rotation found so far) via QRAM lookups. Concretely:

1. **Registers:**
   - $|k\rangle_{\text{idx}}$ ($\lceil \log_2 N \rceil$ qubits) holds the candidate rotation index.
   - $|r\rangle_{\text{ref}}$ ($\lceil \log_2 N \rceil$ qubits) holds a reference rotation index (initially chosen at random).
   - $|\ell\rangle$ ($\lceil \log_2 N \rceil$ qubits) is an offset register.
   - $|c\rangle_{\text{cost}}$ (enough qubits to store an integer up to N) accumulates the cost.
   - Ancilla qubits for arithmetic and QRAM addressing.
2. **Coherent Cost Computation:** To compute $C(k)$, we compare $S^{(k)}$ against $S^{(r)}$ character-wise, using QRAM reads:
   (a) Initialize $|\ell\rangle \leftarrow |0\rangle$, $|c\rangle_{\text{cost}} \leftarrow |0\rangle$.
   (b) For $\ell = 0$ to $N-1$ (in superposition with early exit upon first mismatch):
      i. Compute addresses $(k+\ell) \bmod N$ and $(r+\ell) \bmod N$ into two address registers.
      ii. Perform two QRAM reads:
      $$|(k+\ell) \bmod N\rangle|0\rangle_{\text{data}_1} \xrightarrow{\text{QRAMRead}} |(k+\ell) \bmod N\rangle|s_{(k+\ell) \bmod N}\rangle_{\text{data}_1},$$
      $$|(r+\ell) \bmod N\rangle|0\rangle_{\text{data}_2} \xrightarrow{\text{QRAMRead}} |(r+\ell) \bmod N\rangle|s_{(r+\ell) \bmod N}\rangle_{\text{data}_2}.$$
      iii. Compare $|s_{(k+\ell) \bmod N}\rangle$ vs. $|s_{(r+\ell) \bmod N}\rangle$ using a quantum comparator: if $s_{(k+\ell) \bmod N} \neq s_{(r+\ell) \bmod N}$, deposit penalty δ into $|c\rangle_{\text{cost}}$, then break the loop. Where δ is defined as
      $$\delta = \begin{cases} 0, & s_{(k+\ell) \bmod N} < s_{(r+\ell) \bmod N}, \\ 1, & s_{(k+\ell) \bmod N} > s_{(r+\ell) \bmod N}, \end{cases}$$
      iv. If $s_{(k+\ell) \bmod N} = s_{(r+\ell) \bmod N}$, increment $\ell \leftarrow \ell + 1$ and continue.
   (c) If all N positions match, leave $|c\rangle_{\text{cost}}$ as zero (indicating equality).
3. **Uncompute Ancilla:** After the mismatch loop, uncompute ℓ and the temporary address registers, leaving $|c\rangle_{\text{cost}}$ holding the final cost $C(k)$.
4. **Oracle Action:** We then apply a phase-kickback or amplitude marker based on whether $C(k) < C(r)$. In Dürr–Høyer, this is performed by marking all k such that $C(k)$ is strictly less than the current minimum cost, without explicitly comparing two strings again.

Because each QRAM access costs $O(\log N)$ depth and early exit typically occurs in $O(1)$ steps for nondegenerate inputs, the expected cost-evaluation depth is $O(\log N)$.

4.3 Dürr–Høyer Minimum-Finding Routine

Once the cost-evaluation oracle $\mathcal{O}_{\text{cost}}$ is in place, we apply Dürr–Høyer minimum finding over the N rotation indices:

1. **Initialization:**
   - Pick a random rotation index $r_0 \in \{0, \ldots, N-1\}$ and load $|r_0\rangle$ into the reference register $|r\rangle_{\text{ref}}$.
   - Compute $C(r_0)$ via $\mathcal{O}_{\text{cost}}$ and store it as the current minimum cost.
2. **Amplitude Amplification Loop [7]:** For $t = 1$ to $T = O(\log N)$:
   (a) Prepare $|\psi\rangle = \frac{1}{\sqrt{N}} \sum_{k=0}^{N-1} |k\rangle_{\text{idx}} |r_{t-1}\rangle_{\text{ref}} |0\rangle_{\text{cost}}$.
   (b) Apply $\mathcal{O}_{\text{cost}}$ in superposition to compute $|k\rangle|r_{t-1}\rangle|C(k)\rangle$.
   (c) Use a comparator subroutine to mark (phase-flip) all indices k for which $C(k) < C(r_{t-1})$.
   (d) Perform amplitude amplification (Grover diffusion) over the marked subspace to boost amplitudes of those k.
   (e) Measure the index register to obtain a candidate $\tilde{r}$. Compute $C(\tilde{r})$ classically (or re-invoke $\mathcal{O}_{\text{cost}}$) and update:

$$\begin{cases} r_t \leftarrow \tilde{r}, & \text{if } C(\tilde{r}) < C(r_{t-1}), \\ r_t \leftarrow r_{t-1}, & \text{otherwise.} \end{cases}$$

3. **Termination:** After T iterations, r_T holds LMSR(S) with high probability ($\geq 1 - 1/N$).

4.4 Example Walkthrough

Consider $S = \texttt{banana}$ ($N = 6$). The rotations are:

$$S^{(0)} = \texttt{banana}, \quad S^{(1)} = \texttt{ananab}, \quad S^{(2)} = \texttt{nanaba},$$
$$S^{(3)} = \texttt{anaban}, \quad S^{(4)} = \texttt{nabana}, \quad S^{(5)} = \texttt{abanan}.$$

1. **Initialization:** Suppose $r_0 = 2$ (rotation `nanaba`). Compute $C(2)$ by comparing `nanaba` to itself—cost 0.
2. **First Iteration:**
   - Prepare $\frac{1}{\sqrt{6}} \sum_{k=0}^{5} |k\rangle|r_0 = 2\rangle|0\rangle$.
   - For each k, compute $C(k)$ by comparing $S^{(k)}$ vs. $S^{(2)}$:

- $k = 0$: Compare "banana" vs. "nanaba"—first mismatch at $\ell = 0$: 'b' < 'n' $\implies C(0) = 0$. Marked.
- $k = 1$: Compare "ananab" vs. "nanaba"—mismatch at $\ell = 0$: 'a' < 'n' $\implies C(1) = 0$. Marked.
- $k = 2$: Same string—cost 0. Not marked (equal).
- $k = 3$: "anaban" vs. "nanaba"—mismatch at $\ell = 0$: 'a' < 'n' $\implies C(3) = 0$. Marked.
- $k = 4$: "nabana" vs. "nanaba"—mismatch at $\ell = 1$: 'a' < 'a'? Equal; next $\ell = 2$: 'b' < 'n' $\implies C(4) = 0$. Marked.
- $k = 5$: "abanan" vs. "nanaba"—mismatch at $\ell = 0$: 'a' < 'n' $\implies C(5) = 0$. Marked.

Every $k \neq 2$ is marked because $C(k) < C(2)$. After amplitude amplification, measuring yields one of $\{0, 1, 3, 4, 5\}$. Suppose $\tilde{r} = 5$ (`abanan`).

- Compare $C(5)$ vs. $C(2)$. Since $C(5) = 0 < 0$? Actually, both are zero because we assign zero penalty for any "less-than" event; in practice, we encode a lex-rank integer. Here "abanan" is lex smaller than "nanaba," so we update $r_1 \leftarrow 5$.

3. **Subsequent Iterations:** Now $r_1 = 5$ (`abanan`). No other rotation yields a smaller cost, so the marked set is empty. The reference remains $r_1 = 5$.

4. **Output:** After $T = O(\log 6) \approx 3$ iterations, the algorithm outputs $r_T = 5$, matching the classical result.

5 QRAM Optimizations

QRAM implementation details critically affect circuit depth and error rates. We discuss two primary approaches: binary tree bucket-brigade QRAM [9] and fault-tolerant QRAM designs with practical resource estimates [10]. In both cases, the goal is to reduce the overhead for indexing string positions in superposition.

5.1 Bucket-Brigade QRAM

The bucket-brigade design implements a binary tree of controlled switches. Each level of the tree selects one branch based on an address qubit, ultimately routing a data-qubit line to the QRAM output register. For an N-element memory, the depth is $O(\log N)$ for a single access, and the width requires $O(N)$ physical qubits (data plus switch nodes). Advantages include straightforward circuit structure and guaranteed $O(\log N)$ depth, but error-correction overhead scales with the number of switch nodes. Recent analyses have examined the resilience of bucket-brigade QRAM to generic noise models [11].

5.2 Hashing-Based QRAM

An alternative uses a two-stage hashing scheme: store data in $M = O(N)$ buckets in classical memory, and implement a quantum hash function $h(i)$ that maps address i to a bucket index $b = h(i)$. A second-level table within each bucket resolves collisions. By storing bucket pointers in a quantum register, one can fetch data in $O(\log N + d)$ depth, where d is the depth of the hash-table lookup. Although the worst-case depth may be $O(\sqrt{N})$ if collisions degrade performance, in expectation—assuming a 2-universal hash function—the depth remains $O(\log N)$ [10]. The trade-off is additional classical preprocessing and possible garbage qubit management.

5.3 Impact on CMP Cost

Each CMP(i, j) requires up to m QRAM reads (where m is the first index where $s_{(i+m) \bmod N} \neq s_{(j+m) \bmod N}$). In the worst case $m = N$. Under bucket-brigade QRAM, this is $O(N \log N)$ depth per CMP. Under hashing-based QRAM with low collision probability, the expected depth per CMP becomes $O(\mathrm{E}[m] \log N)$, where $\mathrm{E}[m]$ for random strings approaches a small constant due to early-exit comparisons. In practice, random text or randomized boundary codes exhibit a first mismatch on average at $O(1)$, leading to $O(\log N)$ expected CMP cost.

6 Analysis

We analyze the algorithm's runtime, success probability, and circuit resources under idealized QRAM assumptions. Recall that the minimum-finding subroutine by Dürr and Høyer invokes amplitude amplification [7] for $O(\sqrt{N})$ Grover oracle calls per iteration, repeated $O(\log N)$ times to boost success probability to $1 - O(1/N)$.

6.1 Worst-Case Runtime

In the worst case (e.g., periodic strings where all rotations are identical until the final character), each CMP must compare all N characters before detecting a mismatch or confirming equality. Each character comparison involves two QRAM accesses, each costing $O(\log N)$ depth. Thus, a single CMP is $O(N \log N)$. A single Grover search iteration requires $O(\sqrt{N})$ such comparisons, giving $O\big(N^{3/2} \log N\big)$ per iteration. Repeating for $O(\log N)$ iterations yields a total worst-case depth of

$$T_{\text{worst}}(N) = O\big(N^{3/2} \log^2 N\big).$$

6.2 Average-Case Runtime

For random strings (or boundary codes with sufficient entropy), the expected mismatch index m is $O(1)$. Consequently, each CMP requires $O(\log N)$ depth on average. The Grover subroutine thus runs in $O(\sqrt{N} \cdot \log N)$ depth per iteration, and

$O(\log N)$ iterations yield

$$T_{\text{avg}}(N) = O\big(\sqrt{N}\log^2 N\big).$$

When searching canonical rotations of benzenoid boundary codes—whose symbols cycle through small alphabets—the probability of early mismatches remains high, supporting near-average-case behavior.

6.3 Qubit and Gate Resources

The resources needed for the algorithm include quantum registers, QRAM, and additional work qubits.

Two $\lceil \log_2 N \rceil$-qubit index registers for CMP comparisons (i and j), one $\lceil \log_2 N \rceil$-qubit offset register ℓ, and one comparison-flag qubit. Total: $2\lceil \log_2 N \rceil + \lceil \log_2 N \rceil + 1 = 3\lceil \log_2 N \rceil + 1$ qubits, plus ancillae for arithmetic and QRAM addressing.

QRAM memory requires N qubits, each storing $\lceil \log_2 |\Sigma| \rceil$ bits (e.g., 2–3 qubits per character for small alphabets). This gives a total data qubits: $N \cdot \lceil \log_2 |\Sigma| \rceil$.

Bucket-brigade QRAM requires $O(N)$ switch qubits. Hashing-based schemes require additional ancilla for hash operations ($O(\log N)$ qubits).

Gate counts scale accordingly: each QRAM access has $O(\log N)$ two-qubit gates, each comparator has $O(1)$ Toffoli gates per character comparison, and arithmetic additions modulo N cost $O(\log N)$ gates. The overall gate complexity thus matches the depth analysis.

6.4 Success Probability

Amplitude amplification reduces failure probability exponentially with the number of Grover iterations. Setting iterations to $O(\log N)$ yields success probability $\geq 1 - O(1/N)$. End-to-end measurement of minReg after the final iteration produces the correct rotation index with high confidence. Error sources include QRAM routing errors and comparator misfires [11]; under an ideal error-free QRAM, the algorithm succeeds with arbitrarily high probability by boosting iterations.

7 Results and Validation

Because current hardware does not support a scalable QRAM implementation, we validate our quantum LMSR algorithm via simulation on the Qiskit Python framework [23] (version 1.2.4) running under Anaconda Python 3.12.2. In each test, we focus solely on correctness—verifying that the returned index indeed produces the lexicographically minimal rotation—without measuring runtime performance.

We completed a series of unit tests with fixed values. In addition, a collection of randomized unit tests was run. The "base" test cases consist of a short list (values from 1–6). For each test, we run the full quantum-circuit simulation, extract the

measured index, and confirm that rotating the input list by that index yields the lexicographically smallest sequence.

A few example unit tests are presented below.

- **Test 0:**
  Input:`[1,3,1,4,1,1]`, Result: 4
  After Rotation by 4: `[1,1,1,3,1,4]`
- **Test 1:**
  Input: `[1,1,2,1,1,1,4]`, Result: 3
  After Rotation by 3: `[1,1,1,4,1,1,2]`
- **Test 2:**
  Input: `[1,3,1,4,1,6]`, Result: 0
  After Rotation by 0: `[1,3,1,4,1,6]`
- **Test 3:**
  Input: `[2,6,2,2,1,6,4,2,5,1,3]`, Result: 9,
  After Rotation by 9: `[1,3,2,6,2,2,1,6,4,2,5]`
- **Test 4:**
  Input: `[1,1,1,1,1,1,1,1]`, Result: 0
  After Rotation by 0: `[1,1,1,1,1,1,1,1]`

Next, we perform property-based testing on randomly generated lists of length between 20 and 60, with values in $\{1, \ldots, 6\}$. For each random list, we recorded the index output by the quantum simulation and then verified that rotating the list by this index yields the lexicographically minimal arrangement. Our algorithm successfully passes all our randomized tests.

7.1 Results

All unit tests—both fixed "base" cases and randomized property-based cases—returned the correct rotation index. Each rotated list was confirmed (by a classical check) to be lexicographically minimal. This establishes the functional correctness of our QRAM-based Dürr–Høyer implementation for LMSR under simulation.

The performance of the algorithm is heavily dependent on QRAM development. Bucket-brigade QRAM adds constant factors that dominate small-N performance. Hashing-based QRAM could reduce constants but requires careful collision management. In practical applications (random text or BECs), expected mismatch positions occur early, yielding $O(\sqrt{N}\log^2 N)$ performance. Worst-case degenerate strings (e.g., $S = a^N$) force $O(N^{3/2}\log^2 N)$. For large benzenoid ensembles, quantum LMSR can accelerate canonical code generation, enabling faster isomer lookup and subgraph enumeration in databases.

The algorithms presented here have been verified using simulations in IBM's Qiskit [23] and supporting Python code. This shows the potential of the provided algorithms. Although QRAM is still primarily theoretical, our analysis shows that the current QRAM models will provide efficiency improvement. It remains for viable hardware to be produced to realize these efficiency improvements.

8 Conclusions and Future Work

We presented a QRAM-accelerated quantum algorithm for the Lexicographically Minimal String Rotation problem, achieving an expected runtime of $O\left(\sqrt{N}\log^2 N\right)$ under early-exit assumptions. By simulating both bucket-brigade and hashing-based QRAM, we demonstrated that the quantum approach can surpass classical benchmarks for sufficiently large N, particularly in canonicalizing benzenoid boundary codes. While near-term quantum hardware remains limited in qubit count and QRAM reliability, our analysis provides a roadmap for future implementations once scalable QRAM becomes available.

There is significant potential for future improvements to this work as hardware improves. Alternative QRAM designs can provide improvements. QRAM error-correction schemes can be tailored for string-based memory, minimizing overhead for address switching and data access [9, 11]. The circuit designs used could be optimized for quantum lexicographic comparators using parallelized early-exit checks and approximate string matching to reduce circuit depth [21].

There is a relationship between the LMSR problem and graph isomorphism. It may be possible to extend QRAM-accelerated rotation search to general graph canonicalization problems, e.g., canonical labeling of molecular graphs beyond benzenoids [22]. With larger string sizes it may be necessary to explore hybrid classical-quantum LMSR, where classical Booth's algorithm narrows candidate ranges, and quantum search refines local minima, reducing QRAM requirements.

Under full QRAM assumptions, our algorithm is asymptotically superior to classical LMSR, but hardware constraints currently limit practical advantage. As QRAM architectures mature and gate fidelities improve, quantum LMSR will become a viable tool for large-scale pattern matching and cheminformatics workflows.

References

[1] K. S. Booth, "Lexicographically least circular substrings," *Information Processing Letters*, vol. 10, no. 4–5, pp. 240–242, 1980.

[2] Y. Shiloach, "Fast canonization of circular strings," *Journal of Algorithms*, vol. 2, no. 2, pp. 107–121, 1981.

[3] L. K. Grover, "A fast quantum mechanical algorithm for database search," in *Proceedings of the 28th Annual ACM Symposium on Theory of Computing (STOC)*, 1996, pp. 212–219.

[4] P. W. Shor, "Algorithms for quantum computation: Discrete logarithms and factoring," in *Proceedings of the 35th Annual Symposium on Foundations of Computer Science (FOCS)*, 1994, pp. 124–134.

[5] M. Boyer, G. Brassard, P. Høyer, and A. Tapp, "Tight bounds on quantum searching," *Fortschritte der Physik*, vol. 46, no. 4–5, pp. 493–505, 1998.

[6] C. Dürr and P. Høyer, "A quantum algorithm for finding the minimum," arXiv preprint quant-ph/9607014, 1996.

[7] G. Brassard, P. Høyer, M. Mosca, and A. Tapp, "Quantum amplitude amplification and estimation," in *Quantum Computation and Information*, vol. 305 of *Contemporary Mathematics*, AMS, 2002, pp. 53–74.

[8] V. Giovannetti, S. Lloyd, and L. Maccone, "Quantum random access memory," *Physical Review Letters*, vol. 100, no. 16, p. 160501, 2008.

[9] V. Giovannetti, S. Lloyd, and L. Maccone, "Architectures for a quantum random access memory," *Physical Review A*, vol. 78, no. 5, p. 052310, 2008.

[10] O. Di Matteo, V. Gheorghiu, and M. Mosca, "Fault-tolerant resource estimation of quantum random-access memories," *IEEE Transactions on Quantum Engineering*, vol. 1, pp. 1–13, 2020.

[11] C. T. Hann, G. Lee, S. M. Girvin, and L. Jiang, "Resilience of quantum random access memory to generic noise," *PRX Quantum*, vol. 2, no. 2, p. 020311, 2021.

[12] H. Ramesh and V. Vinay, "String matching in $\tilde{O}(\sqrt{n} + \sqrt{m})$ quantum time," *Journal of Discrete Algorithms*, vol. 1, no. 1, pp. 103–110, 2003.

[13] A. Montanaro, "Quantum pattern matching fast on average," *Algorithmica*, vol. 77, no. 1, pp. 16–39, 2017.

[14] A. Ambainis, "Quantum walk algorithm for element distinctness," *SIAM Journal on Computing*, vol. 37, no. 1, pp. 210–239, 2007.

[15] C. Yuan and M. Carbin, "Tower: Data structures in quantum superposition," arXiv preprint arXiv:2205.10255, 2022.

[16] M. A. Nielsen and I. L. Chuang, *Quantum Computation and Quantum Information*, 10th Anniversary ed. Cambridge University Press, 2010.

[17] I. Gutman and S. J. Cyvin, *Introduction to the Theory of Benzenoid Hydrocarbons*. Berlin: Springer-Verlag, 1989.

[18] I. Gutman and O. E. Polansky, *Mathematical Concepts in Organic Chemistry*. Berlin: Springer-Verlag, 1986.

[19] S. J. Cyvin and I. Gutman, *Kekulé Structures in Benzenoid Hydrocarbons*, Lecture Notes in Chemistry, vol. 46. Berlin: Springer-Verlag, 1988.

[20] Y. Cao, J. Romero, J. P. Olson, M. Degroote, P. D. Johnson, M. Kieferová, I. D. Kivlichan, T. Menke, B. Peropadre, N. P. D. Sawaya, S. Sim, L. Veis, and A. Aspuru-Guzik, "Quantum chemistry in the age of quantum computing," *Chemical Reviews*, vol. 119, no. 19, pp. 10856–10915, 2019.

[21] D. S. Oliveira and R. V. Ramos, "Quantum bit string comparator: Circuits and applications," *Quantum Computers and Computing*, vol. 7, no. 1, pp. 17–26, 2007.

[22] B. L. Douglas and J. B. Wang, "A classical approach to the graph isomorphism problem using quantum walks," *Journal of Physics A: Mathematical and Theoretical*, vol. 41, no. 7, p. 075303, 2008.

[23] Qiskit contributors, "Qiskit: An open-source framework for quantum computing," 2024. [Online]. Available: `https://doi.org/10.5281/zenodo.2573505`